

```
// =====
// 文件: main.dart
// 原始行数: 99 | 保留行数: 99
// =====
import 'dart:async';
import 'dart:ui';

import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:flutter_localizations/flutter_localizations.dart';
import 'package:provider/provider.dart';

import 'core/theme/app_theme.dart';
import 'presentation/pages/home/home_page.dart';
import 'presentation/providers/bill_provider.dart';
import 'presentation/providers/points_provider.dart';
import 'services/api_service.dart';
import 'services/auth_service.dart';
import 'services/notification_service.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  FlutterError.onError = (FlutterErrorDetails details) {
    FlutterError.presentError(details);
    debugPrint('[main] FlutterError: ${details.exceptionAsString()}');
  };

  PlatformDispatcher.instance.onError = (Object error, StackTrace stack) {
    debugPrint('[main] Unhandled zone error: $error');
    debugPrint('$stack');
    return true;
  };

  SystemChrome.setEnabledSystemUIMode(SystemUiMode.edgeToEdge);
  SystemChrome.setSystemUIOverlayStyle(
    const SystemUiOverlayStyle(
      statusBarColor: Colors.transparent,
      statusBarIconBrightness: Brightness.dark,
      statusBarBrightness: Brightness.light,
      systemNavigationBarColor: Colors.transparent,
      systemNavigationBarIconBrightness: Brightness.dark,
    ),
  );

  await _safeInit('ApiService.initDeviceId', ApiService.initDeviceId);
  await _safeInit('AuthService.init', AuthService.init);

  runApp(
    MultiProvider(
```

```

        providers: <ChangeNotifierProvider<dynamic>>[
          ChangeNotifierProvider<BillProvider>(
            create: (_) => BillProvider(),
          ),
          ChangeNotifierProvider<PointsProvider>(
            create: (_) => PointsProvider(),
          ),
        ],
        child: const AirMoneyApp(),
      ),
    );

    WidgetsBinding.instance.addPostFrameCallback((_) {
      unawaited(_safeInit('NotificationService.init', NotificationService.init));
    });
  }

  Future<void> _safeInit(String name, Future<void> Function() task) async {
    try {
      await task();
    } catch (error, stack) {
      debugPrint('[main] $name failed: $error');
      debugPrint('$stack');
    }
  }
}

class AirMoneyApp extends StatelessWidget {
  const AirMoneyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: '哎呀，钱!',
      debugShowCheckedModeBanner: false,
      theme: AppTheme.lightTheme,
      darkTheme: AppTheme.darkTheme,
      themeMode: ThemeMode.system,
      locale: const Locale('zh', 'CN'),
      localizationsDelegates: const <LocalizationsDelegate<dynamic>>[
        GlobalMaterialLocalizations.delegate,
        GlobalWidgetsLocalizations.delegate,
        GlobalCupertinoLocalizations.delegate,
      ],
      supportedLocales: const <Locale>[
        Locale('zh', 'CN'),
        Locale('zh'),
        Locale('en', 'US'),
        Locale('en'),
      ],
      home: const HomePage(),
    );
  }
}

```

```

    );
  }
}

// =====
// 文件: data/models/bill.dart
// 原始行数: 92 | 保留行数: 91
// =====
/// 账单类型: 支出 / 收入
enum BillType { expense, income }

extension BillTypeExt on BillType {
  String get value => this == BillType.expense ? 'expense' : 'income';
  bool get isExpense => this == BillType.expense;
}

class Bill {
  final int? id;
  final double amount;
  final String category;
  final String note;
  final String payMethod;
  final DateTime date;
  final DateTime createdAt;

  /// 支出(expense) 或 收入(income)
  final BillType type;

  Bill({
    this.id,
    required this.amount,
    required this.category,
    this.note = '',
    this.payMethod = '其他',
    required this.date,
    DateTime? createdAt,
    this.type = BillType.expense,
  }) : createdAt = createdAt ?? DateTime.now();

  bool get isExpense => type == BillType.expense;
  bool get isIncome => type == BillType.income;

  Map<String, dynamic> toMap() {
    return {
      if (id != null) 'id': id,
      'amount': amount,
      'category': category,
      'note': note,
      'pay_method': payMethod,
      'date': date.toIso8601String().substring(0, 10),
    };
  }
}

```

```

        'created_at': createdAt.toIso8601String(),
        'type': type.value,
    };
}

static BillType _parseType(dynamic raw) {
    final t = (raw?.toString() ?? '').trim().toLowerCase();
    if (t == 'income' || t == '\u6536\u5165' || t == 'in' || t == 'credit') {
        return BillType.income;
    }
    return BillType.expense;
}

factory Bill.fromMap(Map<String, dynamic> map) {
    final t = _parseType(map['type']);
    return Bill(
        id: map['id'] as int?,
        amount: (map['amount'] as num).toDouble(),
        category: map['category'] as String,
        note: map['note'] as String? ?? '',
        payMethod: map['pay_method'] as String? ?? '其他',
        date: DateTime.parse(map['date'] as String),
        createdAt: DateTime.parse(map['created_at'] as String),
        type: t,
    );
}

Bill copyWith({
    int? id,
    double? amount,
    String? category,
    String? note,
    String? payMethod,
    DateTime? date,
    DateTime? createdAt,
    BillType? type,
}) {
    return Bill(
        id: id ?? this.id,
        amount: amount ?? this.amount,
        category: category ?? this.category,
        note: note ?? this.note,
        payMethod: payMethod ?? this.payMethod,
        date: date ?? this.date,
        createdAt: createdAt ?? this.createdAt,
        type: type ?? this.type,
    );
}
}

```

```
// =====
// 文件: data/models/consult_session.dart
// 原始行数: 85 | 保留行数: 85
// =====
/// 买前咨询会话
class ConsultSession {
  final String id;
  final String title; // 首条用户消息摘要, 用于列表展示
  final List<ConsultMessage> messages;
  final DateTime updatedAt;

  ConsultSession({
    required this.id,
    required this.title,
    required this.messages,
    required this.updatedAt,
  });

  ConsultSession copyWith({
    String? id,
    String? title,
    List<ConsultMessage>? messages,
    DateTime? updatedAt,
  }) =>
    ConsultSession(
      id: id ?? this.id,
      title: title ?? this.title,
      messages: messages ?? this.messages,
      updatedAt: updatedAt ?? this.updatedAt,
    );

  Map<String, dynamic> toJson() => {
    'id': id,
    'title': title,
    'messages': messages.map((m) => m.toJson()).toList(),
    'updatedAt': updatedAt.toIso8601String(),
  };

  static ConsultSession fromJson(Map<String, dynamic> json) => ConsultSession(
    id: json['id'] as String? ?? '',
    title: json['title'] as String? ?? '新对话',
    messages: (json['messages'] as List<dynamic>?)
      ?.map((e) => ConsultMessage.fromJson(Map<String, dynamic>.from(e as Map)))
      .toList() ??
    [],
    updatedAt: DateTime.tryParse(json['updatedAt'] as String? ?? '') ?? DateTime.now(),
  );
}

/// 智能体步骤 (分析意图、查支出等, 可展示)
```

```

class AgentStep {
  final String label;
  final String? result;

  const AgentStep(this.label, [this.result]);

  Map<String, dynamic> toJson() => {'label': label, if (result != null) 'result': result};
  static AgentStep fromJson(Map<String, dynamic> j) => AgentStep(
    j['label'] as String? ?? '',
    j['result'] as String?,
  );
}

class ConsultMessage {
  final String role; // user | assistant
  final String content;
  final String? reasoning; // 思考过程, 仅 assistant
  final List<AgentStep>? agentSteps; // 智能体步骤, 仅 assistant 首条

  ConsultMessage({required this.role, required this.content, this.reasoning, this.agentSteps});

  Map<String, dynamic> toJson() => {
    'role': role,
    'content': content,
    if (reasoning != null && reasoning!.isNotEmpty) 'reasoning': reasoning,
    if (agentSteps != null && agentSteps!.isNotEmpty)
      'agentSteps': agentSteps!.map((s) => s.toJson()).toList(),
  };

  static ConsultMessage fromJson(Map<String, dynamic> json) => ConsultMessage(
    role: json['role'] as String? ?? 'user',
    content: json['content'] as String? ?? '',
    reasoning: json['reasoning'] as String?,
    agentSteps: (json['agentSteps'] as List<dynamic>?)
      ?.map((e) => AgentStep.fromJson(Map<String, dynamic>.from(e as Map)))
      .toList(),
  );
}

// =====
// 文件: data/repositories/bill_repository.dart
// 原始行数: 84 | 保留行数: 84
// =====
import '../bill_storage.dart';
import '../database.dart';
import '../models/bill.dart';

class BillRepository {
  BillStorage get _storage => createBillStorage();
}

```

```

bool _matchesType(Bill bill, BillType? type) {
    if (type == null) return true;
    return bill.type == type;
}

Future<int> insert(Bill bill) async {
    return _storage.insert(bill);
}

Future<int> updateById(int id, Bill bill) async {
    return _storage.updateById(id, bill);
}

Future<int> deleteById(int id) async {
    return _storage.deleteById(id);
}

Future<List<Bill>> getBillsInRange(
    DateTime start,
    DateTime end, {
    BillType? type,
}) async {
    final startStr = start.toIso8601String().substring(0, 10);
    final endStr = end.toIso8601String().substring(0, 10);
    const where = 'date >= ? AND date <= ?';
    final whereArgs = <dynamic>[startStr, endStr];
    final maps = await _storage.query(
        where: where,
        whereArgs: whereArgs,
        orderBy: 'date DESC, created_at DESC',
    );
    return maps
        .map((m) => Bill.fromMap(m))
        .where((bill) => _matchesType(bill, type))
        .toList();
}

Future<List<Bill>> getRecentBills({int limit = 20}) async {
    final maps = await _storage.query(orderBy: 'created_at DESC', limit: limit);
    return maps.map((m) => Bill.fromMap(m)).toList();
}

Future<List<Bill>> getAllBills() async {
    final maps = await _storage.query(orderBy: 'created_at DESC');
    return maps.map((m) => Bill.fromMap(m)).toList();
}

Future<double> getTotalInRange(
    DateTime start,

```

```

    DateTime end, {
    BillType? type,
  }) async {
    final bills = await getBillsInRange(start, end, type: type);
    return bills.fold<double>(0, (sum, bill) => sum + bill.amount);
  }

Future<Map<String, double>> getCategoryTotalsInRange(
  DateTime start,
  DateTime end, {
  BillType type = BillType.expense,
}) async {
  final map = <String, double>{};
  final bills = await getBillsInRange(start, end, type: type);
  for (final bill in bills) {
    final category = bill.category.trim();
    if (category.isEmpty) continue;
    map[category] = (map[category] ?? 0) + bill.amount;
  }
  return map;
}

Future<Bill?> getLatestBill() async {
  final list = await getRecentBills(limit: 1);
  return list.isEmpty ? null : list.first;
}
}

// =====
// 文件: data/database_io.dart
// 原始行数: 89 | 保留行数: 89
// =====
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';
import 'bill_storage.dart';
import 'models/bill.dart';

BillStorage createBillStorage() => _SqfliteBillStorage();

class _SqfliteBillStorage implements BillStorage {
  Database? _db;

  Future<Database> get _database async {
    if (_db != null) return _db!;
    final dir = await getDatabasesPath();
    final dbPath = join(dir, 'airmoney.db');
    _db = await openDatabase(
      dbPath,
      version: 2,

```



```

onCreate: (db, version) async {
    await db.execute('''
        CREATE TABLE bills (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            amount REAL NOT NULL,
            category TEXT NOT NULL,
            note TEXT,
            pay_method TEXT,
            date TEXT NOT NULL,
            created_at TEXT NOT NULL,
            type TEXT DEFAULT 'expense'
        )
    ''');
    await db.execute('CREATE INDEX idx_bills_date ON bills(date)');
},
onUpgrade: (db, oldVersion, newVersion) async {
    if (oldVersion < 2 && newVersion >= 2) {
        await db.execute(
            "ALTER TABLE bills ADD COLUMN type TEXT DEFAULT 'expense'",
        );
    }
},
);
return _db!;
}

@override
Future<int> insert(Bill bill) async {
    final db = await _database;
    return db.insert('bills', bill.toMap());
}

@override
Future<int> updateById(int id, Bill bill) async {
    final db = await _database;
    final map = bill.toMap()..remove('id');
    return db.update('bills', map, where: 'id = ?', whereArgs: [id]);
}

@override
Future<int> deleteById(int id) async {
    final db = await _database;
    return db.delete('bills', where: 'id = ?', whereArgs: [id]);
}

@override
Future<List<Map<String, dynamic>>> query({
    String? where,
    List<dynamic>? whereArgs,
    String? orderBy,

```

```

        int? limit,
    }) async {
        final db = await _database;
        return db.query(
            'bills',
            where: where,
            whereArgs: whereArgs,
            orderBy: orderBy,
            limit: limit,
        );
    }

    @override
    Future<List<Map<String, dynamic>>> rawQuery(
        String sql, [
        List<dynamic>? args,
    ]) async {
        final db = await _database;
        return db.rawQuery(sql, args);
    }
}

// =====
// 文件: data/database_web.dart
// 原始行数: 190 | 保留行数: 190
// =====
import 'dart:convert';
import 'package:flutter/foundation.dart';
import 'package:shared_preferences/shared_preferences.dart';
import 'models/bill.dart';
import 'bill_storage.dart';

BillStorage createBillStorage() => _WebBillStorage();

class _WebBillStorage implements BillStorage {
    static const _key = 'airmoney_bills';

    List<Map<String, dynamic>> _bills = [];
    bool _loaded = false;

    Future<void> _ensureLoaded() async {
        if (_loaded) return;
        try {
            final prefs = await SharedPreferences.getInstance();
            final json = prefs.getString(_key);
            if (json != null && json.isNotEmpty) {
                _bills = List<Map<String, dynamic>>.from(
                    (jsonDecode(json) as List).map(
                        (e) => Map<String, dynamic>.from(e as Map),
                    ),
                );
            }
        } catch (e) {
            // ignore
        }
    }
}

```

```

        ),
    );
}
} catch (e) {
    debugPrint('[WebBillStorage] load error: $e');
}
_loaded = true;
}

Future<void> _persist() async {
    try {
        final prefs = await SharedPreferences.getInstance();
        await prefs.setString(_key, jsonEncode(_bills));
    } catch (e) {
        debugPrint('[WebBillStorage] persist error: $e');
    }
}

@override
Future<int> insert(Bill bill) async {
    await _ensureLoaded();
    final nextId = _bills.isEmpty
        ? 1
        : (_bills
            .map((m) => m['id'] as int? ?? 0)
            .reduce((a, b) => a > b ? a : b) +
            1);
    final m = bill.toMap();
    m['id'] = nextId;
    if (!m.containsKey('type')) m['type'] = 'expense';
    _bills.add(m);
    await _persist();
    return nextId;
}

@override
Future<int> updateById(int id, Bill bill) async {
    await _ensureLoaded();
    final idx = _bills.indexWhere((m) => (m['id'] as int? ?? -1) == id);
    if (idx < 0) return 0;
    final map = bill.toMap()..remove('id');
    map['id'] = id;
    _bills[idx] = map;
    await _persist();
    return 1;
}

@override
Future<int> deleteById(int id) async {
    await _ensureLoaded();

```

```

    final before = _bills.length;
    _bills.removeWhere((m) => (m['id'] as int? ?? -1) == id);
    final removed = before - _bills.length;
    if (removed > 0) {
        await _persist();
    }
    return removed;
}

@override
Future<List<Map<String, dynamic>>> query({
    String? where,
    List<dynamic>? whereArgs,
    String? orderBy,
    int? limit,
}) async {
    await _ensureLoaded();
    var list = List<Map<String, dynamic>>.from(_bills);
    if (where != null && whereArgs != null && whereArgs.isNotEmpty) {
        if (where.contains('date')) {
            final startStr = whereArgs.isNotEmpty ? whereArgs[0].toString() : '';
            final endStr = whereArgs.length > 1 ? whereArgs[1].toString() : '';
            list = list.where((m) {
                final d = m['date'] as String? ?? '';
                return (startStr.isEmpty || d.compareTo(startStr) >= 0) &&
                    (endStr.isEmpty || d.compareTo(endStr) <= 0);
            }).toList();
        }
        if (where.contains('type')) {
            final typeVal = whereArgs.length >= 3 ? whereArgs[2].toString() : null;
            if (typeVal != null) {
                list = list
                    .where((m) => (m['type'] as String? ?? 'expense') == typeVal)
                    .toList();
            }
        }
    }
    if (orderBy != null) {
        if (orderBy.contains('created_at') && orderBy.contains('DESC')) {
            list.sort(
                (a, b) => (b['created_at'] ?? '').toString().compareTo(
                    (a['created_at'] ?? '').toString(),
                ),
            );
        } else if (orderBy.contains('date') && orderBy.contains('DESC')) {
            list.sort(
                (a, b) => (b['date'] ?? '').toString().compareTo(
                    (a['date'] ?? '').toString(),
                ),
            );
        }
    }
}

```

```

    }
  }
  if (limit != null && limit > 0) {
    list = list.take(limit).toList();
  }
  return list;
}

@override
Future<List<Map<String, dynamic>>> rawQuery(
  String sql, [
    List<dynamic>? args,
  ]) async {
  await _ensureLoaded();
  if (sql.contains('SUM(amount)') && sql.contains('WHERE')) {
    final startStr = args != null && args.isNotEmpty
      ? args[0].toString()
      : '';
    final endStr = args != null && args.length > 1 ? args[1].toString() : '';
    final filterType = args != null && args.length >= 3
      ? args[2].toString()
      : null;
    double total = 0;
    for (final m in _bills) {
      final d = m['date'] as String? ?? '';
      if ((startStr.isEmpty || d.compareTo(startStr) >= 0) &&
        (endStr.isEmpty || d.compareTo(endStr) <= 0)) {
        if (filterType != null) {
          final t = m['type'] as String? ?? 'expense';
          if (t != filterType) continue;
        }
        total += (m['amount'] as num?)?.toDouble() ?? 0;
      }
    }
    return [
      {'total': total},
    ];
  }
  if (sql.contains('GROUP BY category')) {
    final startStr = args != null && args.isNotEmpty
      ? args[0].toString()
      : '';
    final endStr = args != null && args.length > 1 ? args[1].toString() : '';
    final filterType = args != null && args.length >= 3
      ? args[2].toString()
      : null;
    final map = <String, double>{};
    for (final m in _bills) {
      final d = m['date'] as String? ?? '';
      if ((startStr.isEmpty || d.compareTo(startStr) >= 0) &&

```

```

        (endStr.isEmpty || d.compareTo(endStr) <= 0)) {
      if (filterType != null) {
        final t = m['type'] as String? ?? 'expense';
        if (t != filterType) continue;
      }
      final cat = m['category'] as String? ?? '其他';
      map[cat] = (map[cat] ?? 0) + ((m['amount'] as num?).toDouble() ?? 0);
    }
  }
  return map.entries
    .map((e) => {'category': e.key, 'total': e.value})
    .toList();
}
return [];
}
}
}

```

```

// =====
// 文件: data/bill_storage.dart
// 原始行数: 19 | 保留行数: 19
// =====
import 'models/bill.dart';

```

```

/// 账单存储抽象, 用于平台差异化实现
abstract class BillStorage {
  Future<int> insert(Bill bill);
  Future<int> updateById(int id, Bill bill);
  Future<int> deleteById(int id);
  Future<List<Map<String, dynamic>>> query({
    String? where,
    List<dynamic>? whereArgs,
    String? orderBy,
    int? limit,
  });
  Future<List<Map<String, dynamic>>> rawQuery(
    String sql, [
    List<dynamic>? args,
  ]);
}

```

```

// =====
// 文件: core/constants.dart
// 原始行数: 48 | 保留行数: 48
// =====
class AppConstants {
  /// 支出分类
  static const List<String> expenseCategories = [
    '餐饮',

```

```

    '交通',
    '购物',
    '娱乐',
    '居家',
    '医疗',
    '教育',
    '其他',
  ];

  /// 收入分类
  static const List<String> incomeCategories = [
    '工资',
    '奖金',
    '兼职',
    '理财收益',
    '红包',
    '其他',
  ];

  static const List<String> payMethods = [
    '微信',
    '支付宝',
    '现金',
    '银行卡',
    '其他',
  ];

  static const Map<String, String> categoryIcons = {
    '餐饮': 'restaurant',
    '交通': 'directions_car',
    '购物': 'shopping_cart',
    '娱乐': 'movie',
    '居家': 'home',
    '医疗': 'local_hospital',
    '教育': 'school',
    '工资': 'work',
    '奖金': 'emoji_events',
    '兼职': 'handyman',
    '理财收益': 'trending_up',
    '红包': 'redeem',
    '其他': 'more_horiz',
  };
}

// =====
// 文件: core/theme/app_theme.dart
// 原始行数: 65 | 保留行数: 65
// =====
import 'package:flutter/material.dart';

```

```

import 'package:flutter/services.dart';

import 'app_colors.dart';

class AppTheme {
  static ThemeData get lightTheme {
    return ThemeData(
      useMaterial3: true,
      brightness: Brightness.light,
      colorScheme: ColorScheme.fromSeed(
        seedColor: AppColors.primaryGreen,
        primary: AppColors.primaryGreen,
        surface: Colors.white,
        onSurface: AppColors.deepText,
      ),
      scaffoldBackgroundColor: AppColors.backgroundLight,
      appBarTheme: const AppBarTheme(
        backgroundColor: Colors.transparent,
        elevation: 0,
        centerTitle: true,
        systemOverlayStyle: SystemUiOverlayStyle(
          statusBarColor: Colors.transparent,
          statusBarIconBrightness: Brightness.dark,
          statusBarBrightness: Brightness.light,
        ),
        titleTextStyle: TextStyle(
          color: AppColors.deepText,
          fontSize: 18,
          fontWeight: FontWeight.w600,
        ),
      ),
    );
  }

  static ThemeData get darkTheme {
    return ThemeData(
      useMaterial3: true,
      brightness: Brightness.dark,
      colorScheme: ColorScheme.fromSeed(
        seedColor: AppColors.primaryGreen,
        primary: AppColors.primaryGreen,
        surface: Color(0xFF252B28),
        onSurface: Colors.white,
        brightness: Brightness.dark,
      ),
      scaffoldBackgroundColor: AppColors.darkBg,
      appBarTheme: const AppBarTheme(
        backgroundColor: Colors.transparent,
        elevation: 0,
        centerTitle: true,

```



```

        systemOverlayStyle: SystemUiOverlayStyle(
          statusBarColor: Colors.transparent,
          statusBarIconBrightness: Brightness.light,
          statusBarBrightness: Brightness.dark,
        ),
        titleTextStyle: TextStyle(
          color: Colors.white,
          fontSize: 18,
          fontWeight: FontWeight.w600,
        ),
      ),
    );
  }
}

// =====
// 文件: core/theme/app_colors.dart
// 原始行数: 32 | 保留行数: 32
// =====
import 'package:flutter/material.dart';

class AppColors {
  static const Color primaryGreen = Color(0xFF2ECC71);
  static const Color primaryLight = Color(0xFFE8F8F5);
  static const Color primaryDark = Color(0xFF27AE60);
  static const Color neutralGrey = Color(0xFF95A5A6);
  static const Color backgroundLight = Color(0xFFFFF8FBF9);
  static const Color expenseRed = Color(0xFFE74C3C);
  static const Color pointsGold = Color(0xFFFF39C12);
  static const Color deepText = Color(0xFF2C3E50);
  static const Color darkBg = Color(0xFF1A1F1C);

  /// 浅色模式卡片阴影
  static List<BoxShadow> get cardShadowLight => [
    BoxShadow(
      color: Colors.black.withOpacity(0.06),
      blurRadius: 12,
      offset: const Offset(0, 4),
    ),
  ];

  /// 深色模式卡片「发光」
  static List<BoxShadow> get cardShadowDark => [
    BoxShadow(
      color: Colors.black.withOpacity(0.3),
      blurRadius: 8,
      offset: const Offset(0, 2),
    ),
  ];
}

```

```
// =====
// 文件: presentation/providers/bill_provider.dart
// 原始行数: 114 | 保留行数: 114
// =====
import 'package:flutter/foundation.dart';
import '../data/models/bill.dart';
import '../data/repositories/bill_repository.dart';

class BillProvider extends ChangeNotifier {
  final BillRepository _repo = BillRepository();
  List<Bill> _recentBills = [];
  bool _loading = false;

  List<Bill> get recentBills => _recentBills;
  bool get loading => _loading;

  Future<void> loadRecentBills() async {
    _loading = true;
    notifyListeners();
    try {
      _recentBills = await _repo.getRecentBills();
    } finally {
      _loading = false;
      notifyListeners();
    }
  }

  Future<int> addBill(Bill bill) async {
    final id = await _repo.insert(bill);
    await loadRecentBills();
    return id;
  }

  Future<bool> updateBill(Bill bill) async {
    final id = bill.id;
    if (id == null) return false;
    final count = await _repo.updateById(id, bill);
    await loadRecentBills();
    return count > 0;
  }

  Future<bool> deleteBill(int id) async {
    final count = await _repo.deleteById(id);
    await loadRecentBills();
    return count > 0;
  }

  Future<List<Bill>> getAllBills() async {
```

```

    return _repo.getAllBills();
}

Future<double> getTodayExpense() async {
    final now = DateTime.now();
    final start = DateTime(now.year, now.month, now.day);
    final end = start.add(const Duration(days: 1));
    return _repo.getTotalInRange(start, end, type: BillType.expense);
}

Future<double> getWeekExpense() async {
    final now = DateTime.now();
    final weekday = now.weekday;
    final start = DateTime(
        now.year,
        now.month,
        now.day,
    ).subtract(Duration(days: weekday - 1));
    final end = start.add(const Duration(days: 7));
    return _repo.getTotalInRange(start, end, type: BillType.expense);
}

Future<double> getTodayIncome() async {
    final now = DateTime.now();
    final start = DateTime(now.year, now.month, now.day);
    final end = start.add(const Duration(days: 1));
    return _repo.getTotalInRange(start, end, type: BillType.income);
}

Future<double> getWeekIncome() async {
    final now = DateTime.now();
    final weekday = now.weekday;
    final start = DateTime(
        now.year,
        now.month,
        now.day,
    ).subtract(Duration(days: weekday - 1));
    final end = start.add(const Duration(days: 7));
    return _repo.getTotalInRange(start, end, type: BillType.income);
}

Future<List<Bill>> getBillsInRange(
    DateTime start,
    DateTime end, {
    BillType? type,
}) async {
    return _repo.getBillsInRange(start, end, type: type);
}

Future<Map<String, double>> getCategoryTotalsInRange(

```

```

    DateTime start,
    DateTime end, {
    BillType type = BillType.expense,
  }) async {
    return _repo.getCategoryTotalsInRange(start, end, type: type);
  }

  Future<double> getTotalInRange(
    DateTime start,
    DateTime end, {
    BillType? type,
  }) async {
    return _repo.getTotalInRange(start, end, type: type);
  }

  Future<Bill?> getLatestBill() async {
    return _repo.getLatestBill();
  }
}

```

```

// =====
// 文件: presentation/providers/points_provider.dart
// 原始行数: 58 | 保留行数: 58
// =====
import 'dart:async';
import 'package:flutter/foundation.dart';
import 'package:shared_preferences/shared_preferences.dart';
import '../services/api_service.dart';
import '../services/auth_service.dart';

class PointsProvider extends ChangeNotifier {
  static const _kBalance = 'points_balance';

  int _balance = 0;
  bool _syncing = false;

  int get balance => _balance;
  bool get syncing => _syncing;

  Future<void> syncFromServer() async {
    if (_syncing) return;
    _syncing = true;
    notifyListeners();
    try {
      final b = await ApiService.initPoints(
        authToken: AuthService.token.isNotEmpty ? AuthService.token : null,
      );
      _balance = b;
      final prefs = await SharedPreferences.getInstance();
    }
  }
}

```

```

    await prefs.setInt(_kBalance, b);
  } catch (e) {
    debugPrint('[Points] sync error: $e');
    final prefs = await SharedPreferences.getInstance();
    _balance = prefs.getInt(_kBalance) ?? 0;
  } finally {
    _syncing = false;
    notifyListeners();
  }
}

Future<void> load() async {
  final prefs = await SharedPreferences.getInstance();
  _balance = prefs.getInt(_kBalance) ?? 0;
  notifyListeners();
  unawaited(syncFromServer());
}

void setBalance(int value) {
  _balance = value < 0 ? 0 : value;
  notifyListeners();
  SharedPreferences.getInstance().then((prefs) {
    prefs.setInt(_kBalance, _balance);
  });
}

void addPoints(int delta) {
  if (delta != 0) setBalance(_balance + delta);
}

bool get hasPoints => _balance > 0;
}

// =====
// 文件: presentation/pages/home/home_page.dart
// 原始行数: 30 | 保留行数: 30
// =====
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../providers/bill_provider.dart';
import '../providers/points_provider.dart';
import '../tab_home/tab_home_page.dart';

class HomePage extends StatefulWidget {
  const HomePage({super.key});

  @override
  State<HomePage> createState() => _HomePageState();
}

```

```

class _HomePageState extends State<HomePage> {
  @override
  void initState() {
    super.initState();
    WidgetsBinding.instance.addPostFrameCallback((_) {
      context.read<BillProvider>().loadRecentBills();
      context.read<PointsProvider>().load();
    });
  }

  @override
  Widget build(BuildContext context) {
    return const TabHomePage();
  }
}

// =====
// 文件: presentation/pages/tab_home/tab_home_page.dart
// 原始行数: 1428 | 保留行数: 170
// =====
import 'package:flutter/material.dart';
import 'package:flutter_slidable/flutter_slidable.dart';
import 'package:provider/provider.dart';
import 'package:intl/intl.dart';
import '../core/theme/app_colors.dart';
import '../core/constants.dart';
import '../data/models/bill.dart';
import '../providers/bill_provider.dart';
import '../providers/points_provider.dart';
import '../widgets/wallet_sheet.dart';
import '../widgets/reminder_settings_sheet.dart';
import '../services/notification_service.dart';
import '../add_bill/add_bill_page.dart';
import '../analysis/analysis_page.dart';
import '../consult/consult_page.dart';

class TabHomePage extends StatefulWidget {
  const TabHomePage({super.key});

  @override
  State<TabHomePage> createState() => _TabHomePageState();
}

class _TabHomePageState extends State<TabHomePage> {
  double _todayExpense = 0;
  double _weekExpense = 0;
  double _todayIncome = 0;

```

```

double _weekIncome = 0;
static const int _recentBillsMaxShown = 10;
bool _recentBillsExpanded = false;
bool _billsLoading = false;
List<Bill> _allBills = [];
BillType? _filterType;
String? _filterCategory;
String? _filterPayMethod;
DateTimeRange? _filterDateRange;
final TextEditingController _noteFilterController = TextEditingController();
bool _reminderEnabled = false;
String? _nextReminderTime;

@override
void initState() {
  super.initState();
  _loadTotals();
  _loadReminderPreview();
}

@override
void dispose() {
  _noteFilterController.dispose();
  super.dispose();
}

Future<void> _loadTotals() async {
  final bp = context.read<BillProvider>();
  final todayE = await bp.getTodayExpense();
  final weekE = await bp.getWeekExpense();
  final todayI = await bp.getTodayIncome();
  final weekI = await bp.getWeekIncome();
  if (mounted) {
    setState(() {
      _todayExpense = todayE;
      _weekExpense = weekE;
      _todayIncome = todayI;
      _weekIncome = weekI;
    });
  }
}

Future<void> _loadReminderPreview() async {
  final entries = await NotificationService.getReminderEntries();
  final enabledTimes = entries
    .where((entry) => entry.enabled)
    .map((entry) => entry.time)
    .toList();

  final nextReminder = _findNextReminderTime(enabledTimes);
  if (!mounted) return;

```

```

    setState(() {
      _reminderEnabled = enabledTimes.isNotEmpty;
      _nextReminderTime = nextReminder;
    });
  }

  String? _findNextReminderTime(List<String> times) {
    if (times.isEmpty) return null;
    final now = DateTime.now();
    DateTime? nextDateTime;
    String? nextTime;

    for (final time in times) {
      final parts = time.split(':');
      final hour = parts.isNotEmpty ? int.tryParse(parts[0]) ?? 13 : 13;
      final minute = parts.length > 1 ? int.tryParse(parts[1]) ?? 0 : 0;

      var candidate = DateTime(now.year, now.month, now.day, hour, minute);
      if (!candidate.isAfter(now)) {
        candidate = candidate.add(const Duration(days: 1));
      }

      if (nextDateTime == null || candidate.isBefore(nextDateTime)) {
        nextDateTime = candidate;
        nextTime = '${hour.toString().padLeft(2, '0')}:${minute.toString().padLeft(2, '0')}';
      }
    }

    return nextTime;
  }

  Future<void> _openReminderSettings() async {
    final saved = await ReminderSettingsSheet.show(context);
    if (!mounted) return;
    await _loadReminderPreview();
    if (saved && mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('提醒设置已保存')),
      );
    }
  }

  Future<void> _loadBills() async {
    if (mounted) {
      setState(() => _billsLoading = true);
    }
    try {
      final bills = await context.read<BillProvider>().getAllBills();
      if (!mounted) return;
      setState(() {

```



```

        _allBills = bills;
    });
} finally {
    if (mounted) {
        setState(() => _billsLoading = false);
    }
}
}

Future<void> _refreshAllData() async {
    await Future.wait([
        _loadTotals(),
        context.read<BillProvider>().loadRecentBills(),
        _loadReminderPreview(),
    ]);
}

List<String> get _categoryOptions {
    final base = _filterType == BillType.expense
        ? AppConstants.expenseCategories
        : _filterType == BillType.income
        ? AppConstants.incomeCategories
        : [...AppConstants.expenseCategories, ...AppConstants.incomeCategories];
    final set = <String>{...base};
    for (final bill in _allBills) {
        if (_filterType != null && bill.type != _filterType) continue;
        set.add(bill.category);
    }
    final list = set.toList();
    list.sort();
    return list;
}

List<Bill> get _filteredBills {
    final noteKeyword = _noteFilterController.text.trim().toLowerCase();
    final range = _filterDateRange;
    final start = range?.start;
    final end = range?.end;
    return _allBills.where((bill) {
        if (_filterType != null && bill.type != _filterType) return false;
        if (_filterCategory != null &&
            _filterCategory!.isNotEmpty &&
            bill.category != _filterCategory) {

```

// =====

// 文件: presentation/pages/add_bill/add_bill_page.dart

// 原始行数: 255 | 保留行数: 170

// =====

```

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

```

```

import 'package:intl/intl.dart';
import '../../core/theme/app_colors.dart';
import '../../core/constants.dart';
import '../../data/models/bill.dart';
import '../../providers/bill_provider.dart';

class AddBillPage extends StatefulWidget {
  final Bill? editingBill;

  const AddBillPage({super.key, this.editingBill});

  @override
  State<AddBillPage> createState() => _AddBillPageState();
}

class _AddBillPageState extends State<AddBillPage> {
  final _amountController = TextEditingController();
  final _noteController = TextEditingController();
  BillType _type = BillType.expense;
  late String _category;
  String _payMethod = AppConstants.payMethods.first;
  DateTime _date = DateTime.now();
  bool _saving = false;

  bool get _isEditing => widget.editingBill != null;

  @override
  void initState() {
    super.initState();
    final bill = widget.editingBill;
    if (bill == null) {
      _category = AppConstants.expenseCategories.first;
      return;
    }
    _type = bill.type;
    _category = bill.category;
    _payMethod = bill.payMethod;
    _date = bill.date;
    _amountController.text = bill.amount.toStringAsFixed(
      bill.amount.truncateToDouble() == bill.amount ? 0 : 2,
    );
    _noteController.text = bill.note;
  }

  List<String> get _categories => _type == BillType.expense
    ? AppConstants.expenseCategories
    : AppConstants.incomeCategories;

  @override
  void dispose() {

```

```

    _amountController.dispose();
    _noteController.dispose();
    super.dispose();
}

Future<void> _save() async {
  final amountStr = _amountController.text.trim();
  final amount = double.tryParse(amountStr);
  if (amount == null || amount <= 0) {
    ScaffoldMessenger.of(
      context,
    ).showSnackBar(const SnackBar(content: Text('请输入有效金额')));
    return;
  }
  setState(() => _saving = true);
  try {
    final bill = Bill(
      id: widget.editingBill?.id,
      amount: amount,
      category: _category,
      note: _noteController.text.trim(),
      payMethod: _payMethod,
      date: _date,
      type: _type,
      createdAt: widget.editingBill?.createdAt,
    );
    final bp = context.read<BillProvider>();
    if (_isEditing) {
      await bp.updateBill(bill);
    } else {
      await bp.addBill(bill);
    }
    if (mounted) {
      Navigator.of(context).pop();
    }
  } finally {
    if (mounted) setState(() => _saving = false);
  }
}

void _onTypeChanged(BillType t) {
  setState(() {
    _type = t;
    _category = _categories.first;
  });
}

@override
Widget build(BuildContext context) {
  return Scaffold(

```

```

appBar: AppBar(
  title: Text(_isEditing ? '编辑记录' : '记一笔'),
  leading: IconButton(
    icon: const Icon(Icons.close),
    onPressed: () => Navigator.of(context).pop(),
  ),
),
body: SingleChildScrollView(
  padding: const EdgeInsets.all(20),
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      const Text(
        '类型',
        style: TextStyle(fontSize: 14, color: AppColors.neutralGrey),
      ),
      const SizedBox(height: 8),
      SegmentedButton<BillType>(
        segments: const [
          ButtonSegment(
            value: BillType.expense,
            label: Text('支出'),
            icon: Icon(Icons.trending_down, size: 18),
          ),
          ButtonSegment(
            value: BillType.income,
            label: Text('收入'),
            icon: Icon(Icons.trending_up, size: 18),
          ),
        ],
        selected: {_type},
        onSelectionChanged: (s) => _onTypeChanged(s.first),
      ),
      const SizedBox(height: 24),
      const Text(
        '金额',
        style: TextStyle(fontSize: 14, color: AppColors.neutralGrey),
      ),
      const SizedBox(height: 8),
      TextField(
        controller: _amountController,
        keyboardType: const TextInputType.numberWithOptions(
          decimal: true,
        ),
        style: const TextStyle(fontSize: 32, fontWeight: FontWeight.bold),
        decoration: InputDecoration(
          hintText: '0',
          prefixText: '¥ ',
          border: OutlineInputBorder(
            borderRadius: BorderRadius.circular(12),

```

```

        ),
      ),
    ),
    const SizedBox(height: 24),
    const Text(
      '分类',
      style: TextStyle(fontSize: 14, color: AppColors.neutralGrey),
    ),
    const SizedBox(height: 8),
    Wrap(
      spacing: 8,
      runSpacing: 8,
      children: _categories.map((c) {
        final selected = _category == c;
        return ChoiceChip(
          label: Text(c),
          selected: selected,
          onSelect: (_) => setState(() => _category = c),
        );
      }).toList(),
    ),
  ),
);
// =====
// 文件: presentation/pages/analysis/analysis_page.dart
// 原始行数: 1350 | 保留行数: 170
// =====
import 'package:flutter/material.dart';
import 'package:flutter_slidable/flutter_slidable.dart';
import 'package:intl/intl.dart';
import 'package:provider/provider.dart';

import '../../core/constants.dart';
import '../../core/theme/app_colors.dart';
import '../../data/models/bill.dart';
import '../../services/analysis_service.dart';
import '../../providers/bill_provider.dart';
import '../../providers/points_provider.dart';
import '../../widgets/wallet_sheet.dart';
import '../add_bill/add_bill_page.dart';

class AnalysisPage extends StatefulWidget {
  const AnalysisPage({super.key});

  @override
  State<AnalysisPage> createState() => _AnalysisPageState();
}

class _AnalysisPageState extends State<AnalysisPage> {
  final TextEditingController _noteFilterController = TextEditingController();

  List<Bill> _allBills = [];
  bool _loadingBills = false;
  bool _batchAnalyzing = false;

```

```

BillType? _filterType;
String? _filterCategory;
String? _filterPayMethod;
DateTimeRange? _filterDateRange;
String _quickDateRange = 'none';

String? _batchAnalysisResult;
String? _batchAnalysisError;
bool _batchAnalysisVisible = false;
final Map<String, String> _batchAnalysisCache = <String, String>{};

Object? _singleAnalysisBillKey;
bool _singleAnalysisVisible = false;
bool _singleAnalyzing = false;
String? _singleAnalysisResult;
String? _singleAnalysisError;
final Map<Object, String> _singleAnalysisCache = <Object, String>{};

@override
void initState() {
  super.initState();
  _loadBills();
}

@override
void dispose() {
  _noteFilterController.dispose();
  super.dispose();
}

Object _billKey(Bill bill) {
  return bill.id ?? bill.createdAt.microsecondsSinceEpoch;
}

String _buildBatchCacheKey(List<Bill> bills) {
  final parts = bills.map((bill) {
    final idPart =
      bill.id?.toString() ??
      bill.createdAt.microsecondsSinceEpoch.toString();
    final datePart = DateFormat('yyyy-MM-dd').format(bill.date);
    return '$idPart|$datePart|${bill.category}|${bill.amount.toStringAsFixed(2)}';
  }).toList()..sort();
  return parts.join('||');
}

Future<void> _loadBills() async {
  if (mounted) {
    setState(() => _loadingBills = true);
  }
}

```

```

try {
    final bills = await context.read<BillProvider>().getAllBills();
    if (!mounted) return;
    setState(() {
        _allBills = bills;
    });
} finally {
    if (mounted) {
        setState(() => _loadingBills = false);
    }
}
}

Future<void> _refreshAllData() async {
    await Future.wait([
        _loadBills(),
        context.read<BillProvider>().loadRecentBills(),
    ]);
}

List<String> get _categoryOptions {
    final base = _filterType == BillType.expense
        ? AppConstants.expenseCategories
        : _filterType == BillType.income
        ? AppConstants.incomeCategories
        : [...AppConstants.expenseCategories, ...AppConstants.incomeCategories];

    final set = <String>{...base};
    for (final bill in _allBills) {
        if (_filterType != null && bill.type != _filterType) continue;
        set.add(bill.category);
    }
    final list = set.toList();
    list.sort();
    return list;
}

List<String> get _payMethodOptions {
    final set = <String>{...AppConstants.payMethods};
    for (final bill in _allBills) {
        set.add(bill.payMethod);
    }
    final list = set.toList();
    list.sort();
    return list;
}

List<Bill> get _filteredBills {
    final noteKeyword = _noteFilterController.text.trim().toLowerCase();
    final start = _filterDateRange?.start;

```

```

final end = _filterDateRange?.end;

final result = _allBills.where((bill) {
  if (_filterType != null && bill.type != _filterType) return false;

  if (_filterCategory != null && _filterCategory!.isNotEmpty) {
    if (bill.category != _filterCategory) return false;
  }

  if (_filterPayMethod != null && _filterPayMethod!.isNotEmpty) {
    if (bill.payMethod != _filterPayMethod) return false;
  }

  if (noteKeyword.isNotEmpty &&
    !bill.note.toLowerCase().contains(noteKeyword)) {
    return false;
  }

  if (start != null && end != null) {
    final day = DateTime(bill.date.year, bill.date.month, bill.date.day);
    final rangeStart = DateTime(start.year, start.month, start.day);
    final rangeEnd = DateTime(end.year, end.month, end.day);
    if (day.isBefore(rangeStart) || day.isAfter(rangeEnd)) return false;
  }

  return true;
}).toList();

result.sort((a, b) {
  final byDate = b.date.compareTo(a.date);
  if (byDate != 0) return byDate;
  return b.createdAt.compareTo(a.createdAt);
});

return result;
}

List<Bill> get _filteredExpenseBills =>
  _filteredBills.where((bill) => bill.isExpense).toList();

bool get _hasFilters {
  return _filterType != null ||
    (_filterCategory?.isNotEmpty ?? false) ||

// =====
// 文件: presentation/pages/analysis/single_analysis_page.dart
// 原始行数: 180 | 保留行数: 120
// =====
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

```



```

import '../../../core/theme/app_colors.dart';
import '../../../data/models/bill.dart';
import '../../../services/analysis_service.dart';
import '../../../providers/bill_provider.dart';
import '../../../providers/points_provider.dart';

class SingleAnalysisPage extends StatefulWidget {
  final Bill bill;

  const SingleAnalysisPage({super.key, required this.bill});

  @override
  State<SingleAnalysisPage> createState() => _SingleAnalysisPageState();
}

class _SingleAnalysisPageState extends State<SingleAnalysisPage> {
  String? _result;
  String? _error;
  bool _loading = false;

  @override
  void initState() {
    super.initState();
    WidgetsBinding.instance.addPostFrameCallback((_) => _analyze());
  }

  Future<void> _analyze() async {
    final points = context.read<PointsProvider>().balance;
    if (points <= 0) {
      setState(() {
        _error = '积分不足, 请先登录或签到获取积分';
      });
      return;
    }
    setState(() {
      _loading = true;
      _error = null;
    });
    try {
      final bp = context.read<BillProvider>();
      final end = widget.bill.date;
      final start = end.subtract(const Duration(days: 7));
      final catTotals = await bp.getCategoryTotalsInRange(start, end, type: BillType.expense);
      final cat7d = catTotals[widget.bill.category];
      final map = cat7d != null ? {widget.bill.category: cat7d} : null;
      final text = await analyzeSingleBill(widget.bill, categoryTotals7d: map);
      if (mounted) {
        setState(() {
          _result = text;
          _loading = false;
        });
      }
    } catch (e) {
      setState(() {
        _error = e.toString();
      });
    }
  }
}

```

```

    });
    context.read<PointsProvider>().syncFromServer();
  }
} catch (e) {
  if (mounted) {
    setState(() {
      _error = e.toString().replaceAll('Exception:', '').trim();
      _loading = false;
    });
  }
}
}
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('单次分析'),
      leading: IconButton(
        icon: const Icon(Icons.close),
        onPressed: () => Navigator.of(context).pop(),
      ),
    ),
    body: SingleChildScrollView(
      padding: const EdgeInsets.all(20),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          _BillSummary(bill: widget.bill),
          const SizedBox(height: 24),
          if (_loading)
            const Center(
              child: Padding(
                padding: EdgeInsets.all(32),
                child: Column(
                  children: [
                    CircularProgressIndicator(),
                    SizedBox(height: 16),
                    Text('正在分析...'),
                  ],
                ),
            ),
          else if (_error != null)
            Container(
              width: double.infinity,
              padding: const EdgeInsets.all(16),
              decoration: BoxDecoration(
                color: AppColors.expenseRed.withValues(alpha: 0.1),
                borderRadius: BorderRadius.circular(12),

```

```

    ),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text(_error!, style: const TextStyle(color: AppColors.expenseRed)),
        const SizedBox(height: 8),
        TextButton(
          onPressed: _analyze,
          child: const Text('重试'),
        ),
      ],
    ),
  ),
)
else if (_result != null && _result!.isNotEmpty)
  Container(
    width: double.infinity,
    padding: const EdgeInsets.all(16),
    decoration: BoxDecoration(
// =====
// 文件: presentation/pages/consult/consult_page.dart
// 原始行数: 1092 | 保留行数: 170
// =====
import 'dart:async';
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../core/constants.dart';
import '../core/theme/app_colors.dart';
import '../data/models/bill.dart';
import '../data/models/consult_session.dart'
  show AgentStep, ConsultMessage, ConsultSession;
import '../services/api_service.dart';
import '../services/consult_service.dart';
import '../services/consult_session_storage.dart';
import '../providers/bill_provider.dart';
import '../providers/points_provider.dart';
import '../widgets/wallet_sheet.dart';

const List<String> _assistantMetaMarkers = <String>[
  '追问',
  '细节',
  '幽默',
  '理解',
  '认同',
  '共情',
  '过渡',
  '回应',
  '反应',
  '标签',
  '动作',

```

```

    '策略',
    '意图',
    '分析',
    '总结',
    '建议',
    '提问',
    'intent',
    'meta',
];

String _stripAssistantMetaTags(String text) {
  if (text.isEmpty) return text;
  final pattern = RegExp(
    r'[\(■\s*(^(\(■{1,24})\s*(\))■)',
  );
  final cleaned = text.replaceAllMapped(pattern, (m) {
    final inside = (m.group(1) ?? '').replaceAll(RegExp(r'\s+'), '');
    final isMeta = _assistantMetaMarkers.any(
      (k) => inside.toLowerCase().contains(k),
    );
    return isMeta ? '' : (m.group(0) ?? '');
  });
  return cleaned
    .replaceAll(RegExp(r'[ \t]{2,}'), ' ')
    .replaceAll(RegExp(r'\n{3,}'), '\n\n')
    .trimRight();
}

class _IntentAnalysisDecision {
  const _IntentAnalysisDecision({required this.shouldAnalyze, this.category});
  final bool shouldAnalyze;
  final String? category;
}

class _ActiveConsultStreamState {
  _ActiveConsultStreamState({required this.sessionId});
  final String sessionId;
  final Completer<void> stopSignal = Completer<void>();
  Future<void>? runner;
  bool isRunning = true;
  bool stopRequested = false;
  String rawContent = '';
  String content = '';
  String? reasoning;
  List<AgentStep> agentSteps = <AgentStep>[];
  int? pointsBalance;
  String? errorMessage;
  int revision = 0;
  void markChanged() => revision += 1;
  void requestStop() {

```

```

        stopRequested = true;
        if (!stopSignal.isCompleted) stopSignal.complete();
        markChanged();
    }
}

final Map<String, _ActiveConsultStreamState> _activeConsultStreams =
    <String, _ActiveConsultStreamState>{};

class ConsultPage extends StatefulWidget {
    const ConsultPage({super.key});
    @override
    State<ConsultPage> createState() => _ConsultPageState();
}

class _ConsultPageState extends State<ConsultPage> {
    final TextEditingController _controller = TextEditingController();
    final ScrollController _scrollController = ScrollController();
    List<ConsultMessage> _messages = <ConsultMessage>[];
    String _streamingRawContent = '';
    String _streamingContent = '';
    String? _streamingReasoning;
    List<AgentStep> _agentSteps = <AgentStep>[];
    bool _loading = false;
    ConsultSession? _currentSession;
    List<ConsultSession> _sessions = <ConsultSession>[];
    bool _loadingSessions = true;
    String? _lastIntentCategory;
    Timer? _streamSyncTimer;
    int _lastObservedStreamRevision = -1;
    String? _observedStreamSessionId;
    bool _syncingCompletedStream = false;

    @override
    void initState() {
        super.initState();
        ApiService.onPointsBalanceChanged = _onPointsBalanceChanged;
        _loadSessionsAndMessages();
        _streamSyncTimer = Timer.periodic(
            const Duration(milliseconds: 160),
            (_) => _syncStreamingState(),
        );
    }

    @override
    void dispose() {
        ApiService.onPointsBalanceChanged = null;
        _streamSyncTimer?.cancel();
        _controller.dispose();
        _scrollController.dispose();
    }
}

```

```

    super.dispose();
  }

void _onPointsBalanceChanged(int balance) {
  if (!mounted) return;
  context.read<PointsProvider>().setBalance(balance);
}

Future<void> _loadSessionsAndMessages() async {
  await ConsultSessionStorage.load();
  if (!mounted) return;
  final cur = await ConsultSessionStorage.ensureCurrentSession();
  if (!mounted) return;
  setState(() {
    _sessions = ConsultSessionStorage.sessions;
    _currentSession = cur;
    _messages = List<ConsultMessage>.from(cur.messages);
    _lastIntentCategory = _extractLastIntentCategory(_messages);
    _loadingSessions = false;
  });
  await _syncStreamingState(force: true);
  _scrollToBottom();
}

Future<void> _switchSession(ConsultSession session) async {
  final s = await ConsultSessionStorage.switchToSession(session.id);
  if (!mounted || s == null) return;
  setState(() {
    _currentSession = s;
    _messages = List<ConsultMessage>.from(s.messages);
    _lastIntentCategory = _extractLastIntentCategory(_messages);
    _loading = false;
    _streamingRawContent = '';
    _streamingContent = '';
    _streamingReasoning = null;
    _agentSteps = <AgentStep>[];
    _observedStreamSessionId = null;
    _lastObservedStreamRevision = -1;
  });
  await _syncStreamingState(force: true);
  _scrollToBottom();
}

// =====
// 文件: presentation/pages/auth/login_page.dart
// 原始行数: 371 | 保留行数: 120
// =====
import 'dart:async';

```

```

import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:intl/intl.dart';
import 'package:provider/provider.dart';

import '../.../core/theme/app_colors.dart';
import '../.../services/auth_service.dart';
import '../.../providers/points_provider.dart';

class LoginPage extends StatefulWidget {
  const LoginPage({super.key});

  /// 以 BottomSheet 形式展示登录（固定高度，不全屏）
  static Future<bool?> show(BuildContext context) async {
    return showModalBottomSheet<bool>(
      context: context,
      isScrollControlled: true,
      backgroundColor: Colors.transparent,
      builder: (_) => const LoginPage(),
    );
  }

  @override
  State<LoginPage> createState() => _LoginPageState();
}

class _LoginPageState extends State<LoginPage> {
  final _phoneController = TextEditingController();
  final _codeController = TextEditingController();
  bool _sending = false;
  bool _logging = false;
  int _countdown = 0;
  Timer? _timer;
  String? _error;

  @override
  void dispose() {
    _phoneController.dispose();
    _codeController.dispose();
    _timer?.cancel();
    super.dispose();
  }

  bool get _phoneValid =>
    _phoneController.text.replaceAll(RegExp(r'^0-9|'), '').length == 11;

  bool get _codeValid => _codeController.text.trim().length >= 4;

  Future<void> _sendCode() async {
    if (!_phoneValid || _sending || _countdown > 0) return;

```

```

setState(() {
  _sending = true;
  _error = null;
});
final result = await AuthService.sendSmsCode(_phoneController.text.trim());
if (!mounted) return;
setState(() => _sending = false);
if (result.success) {
  setState(() => _countdown = 60);
  _timer?.cancel();
  _timer = Timer.periodic(const Duration(seconds: 1), (t) {
    if (!mounted) {
      t.cancel();
      return;
    }
    setState(() {
      _countdown--;
      if (_countdown <= 0) t.cancel();
    });
  });
} else {
  setState(() => _error = result.error ?? '发送失败');
}
}

Future<void> _showGrantAnimation(BuildContext ctx, int points) async {
  final fmt = NumberFormat('#,###');
  await showGeneralDialog(
    context: ctx,
    barrierDismissible: true,
    barrierLabel: 'grant',
    barrierColor: Colors.black54,
    transitionDuration: const Duration(milliseconds: 350),
    transitionBuilder: (context, anim, __, child) {
      return ScaleTransition(
        scale: CurvedAnimation(parent: anim, curve: Curves.easeOutBack),
        child: FadeTransition(opacity: anim, child: child),
      );
    },
    pageBuilder: (context, __, __) {
      return Center(
        child: Material(
          color: Colors.transparent,
          child: Container(
            width: 260,
            padding: const EdgeInsets.symmetric(vertical: 28, horizontal: 24),
            decoration: BoxDecoration(
              color: Theme.of(context).brightness == Brightness.dark
                ? const Color(0xFF2C2C2E)
                : Colors.white,
            ),
          ),
        ),
      );
    },
  );
}

```



```

        borderRadius: BorderRadius.circular(20),
      ),
      child: Column(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          const Text('■', style: TextStyle(fontSize: 40)),
          const SizedBox(height: 12),
          Text(
            '登录成功',
            style: TextStyle(
              fontSize: 16,
              fontWeight: FontWeight.bold,
              color: Theme.of(context).brightness == Brightness.dark
                ? Colors.white
                : Colors.black87,
            ),
          ),
          const SizedBox(height: 8),
        ],
      ),
    ),
  ),
);

// =====
// 文件: services/api_service.dart
// 原始行数: 401 | 保留行数: 120
// =====
import 'dart:async';
import 'dart:convert';
import 'package:flutter/foundation.dart';
import 'package:http/http.dart' as http;
import 'package:shared_preferences/shared_preferences.dart';
import 'package:device_info_plus/device_info_plus.dart';
import 'package:uuid/uuid.dart';

/// 流式输出块 (含思考内容、正文、积分)
class ChatStreamChunk {
  final String content;
  final String? reasoningContent;
  final bool isReasoning;
  final bool isComplete;
  final int? pointsBalance;

  ChatStreamChunk({
    required this.content,
    this.reasoningContent,
    this.isReasoning = false,
    this.isComplete = false,
    this.pointsBalance,
  });
}

/// 与后端 API 交互 (混元代理、积分、登录)
/// 后端可与 AirRead 共用, 或使用 AirMoney 独立 server

```

```

class ApiService {
  static const _prefDeviceId = 'device_id';

  static String get proxyUrl => const String.fromEnvironment(
    'AIRMONEY_API_PROXY_URL',
    defaultValue: 'http://localhost:9001',
  );

  static String get apiKey =>
    const String.fromEnvironment('AIRMONEY_API_KEY', defaultValue: '');

  static String _deviceId = '';

  static String get deviceId => _deviceId;

  static Future<void> initDeviceId() async {
    if (_deviceId.isNotEmpty) return;
    if (!kIsWeb) {
      try {
        final info = DeviceInfoPlugin();
        if (defaultTargetPlatform == TargetPlatform.android) {
          final android = await info.androidInfo;
          _deviceId = android.id.isNotEmpty ? android.id : android.fingerprint;
        } else if (defaultTargetPlatform == TargetPlatform.iOS) {
          final ios = await info.iOSInfo;
          _deviceId = ios.identifierForVendor ?? '';
        }
      } catch (e) {
        debugPrint('[ApiService] device_info error: $e');
      }
    }
    if (_deviceId.isEmpty) {
      final prefs = await SharedPreferences.getInstance();
      final stored = (prefs.getString(_prefDeviceId) ?? '').trim();
      if (stored.isNotEmpty) {
        _deviceId = stored;
      } else {
        _deviceId = const Uuid().v4();
        await prefs.setString(_prefDeviceId, _deviceId);
      }
    }
  }

  static Map<String, String> headers({String? authToken}) {
    final h = <String, String>{
      'Content-Type': 'application/json; charset=utf-8',
      'X-Device-Id': _deviceId,
    };
    if (apiKey.trim().isEmpty) h['X-API-Key'] = apiKey;
    if (authToken != null && authToken.isNotEmpty) {

```

```

    h['X-Auth-Token'] = authToken;
  }
  return h;
}

/// 调用混元 ChatCompletions (非流式)
static Future<String> chatCompletions({
  required List<Map<String, String>> messages,
  String? authToken,
}) async {
  final url = Uri.parse('$proxyUrl/');
  final payload = {
    'host': 'hunyuan.tencentcloudapi.com',
    'service': 'hunyuan',
    'action': 'ChatCompletions',
    'version': '2023-09-01',
    'region': 'ap-guangzhou',
    'stream': false,
    'payload': {
      'Model': 'hunyuan-2.0-instruct-20251111',
      'Stream': false,
      'Messages': messages
        .map((m) => {'Role': m['role']!, 'Content': m['content'] ?? ''})
        .toList(),
      'EnableEnhancement': true,
      'ForceSearchEnhancement': true,
    },
  };
  final resp = await http
    .post(
      url,
      headers: headers(authToken: authToken),
      body: jsonEncode(payload),
    )
    .timeout(const Duration(seconds: 60));
  if (resp.statusCode != 200) {
    try {
      final err = jsonDecode(resp.body);
      final msg = err['message'] ?? err['error'] ?? resp.body;
      throw ApiException(msg.toString());
    } catch (e) {
      if (e is ApiException) rethrow;
      throw ApiException('请求失败: ${resp.statusCode}');
    }
  }

  // =====
  // 文件: services/analysis_service.dart
  // 原始行数: 104 | 保留行数: 104
  // =====
  import '../data/models/bill.dart';
  import 'api_service.dart';

```

```

import 'auth_service.dart';

String _getCurrentDateInfo() {
  final now = DateTime.now();
  final weekday = ['一', '二', '三', '四', '五', '六', '日'][now.weekday - 1];
  return '${now.year}年${now.month}月${now.day}日（周$weekday）';
}

/// 单次分析：从最近记录中选择一笔进行深度分析
Future<String> analyzeSingleBill(
  Bill bill, {
    Map<String, double>? categoryTotals7d,
  }) async {
  final dateInfo = _getCurrentDateInfo();
  final sysPrompt =
    '''你是「哎呀，钱！」的消费分析顾问。主打「少花点，存多点」。

当前日期：$dateInfo

输入：用户的单笔账单数据

任务：判断这笔是否偏冲动/不必要，给1-2条简短建议和1个反思问题。

要求：
- 语气温和、不指责、带点幽默
- 建议具体、可操作
- 控制字数约100字
- 用「建议」「可以尝试」等措辞
- 涉及时间判断时，以当前日期为准
- 如需查询实时信息（如当前物价、优惠活动等），可联网搜索后给出建议''';

  final catInfo = categoryTotals7d != null && categoryTotals7d.isNotEmpty
    ? '\n该分类近7天汇总：$categoryTotals7d'
    : '';
  final userContent =
    '''本次记录：
- 金额：${bill.amount}元
- 分类：${bill.category}
- 备注：${bill.note.isEmpty ? '无' : bill.note}
- 日期：${bill.date.toString().substring(0, 10)}
$catInfo

请给出分析、建议和反思问题。''';

  final messages = [
    {'role': 'system', 'content': sysPrompt},
    {'role': 'user', 'content': userContent},
  ];
  return ApiService.chatCompletions(
    messages: messages,

```

```

    authToken: AuthService.token.isNotEmpty ? AuthService.token : null,
  );
}

```

/// 周期分析

```

Future<String> analyzePeriod({
  required List<Bill> bills,
  required double total,
  required Map<String, double> categoryTotals,
  required String periodLabel,
}) async {
  final dateInfo = _getCurrentDateInfo();
  final sysPrompt =
    '''你是「哎呀，钱！」的消费分析顾问。主打「少花点，存多点」。

```

当前日期: \$dateInfo

输入: 用户一段时间内的账单数据及统计

任务: 概括消费特点、指出可能的浪费点、给2-3条可执行建议，帮用户反省反省。

要求:

- 语气温和、不指责、带点幽默
- 建议具体、可操作
- 控制字数约200字
- 用「建议」「可以尝试」等措辞
- 涉及时间判断时，以当前日期为准
- 如需查询实时信息（如当前物价、优惠活动等），可联网搜索后给出建议'''

```

    final catLines = categoryTotals.entries
      .map((e) => '- ${e.key} ${e.value.toStringAsFixed(0)}元')
      .join('\n');
    final userContent =
      '''$periodLabel 支出统计:

```

```

- 总额: ${total.toStringAsFixed(0)}元
- 分类分布:
$catLines

```

```

最近几笔: ${bills.take(10).map((b) => '${b.date.toString().substring(0, 10)} ${b.category} ${b.amount}元${

```

请给出分析报告。'''

```

    final messages = [
      {'role': 'system', 'content': sysPrompt},
      {'role': 'user', 'content': userContent},
    ];
    return ApiService.chatCompletions(
      messages: messages,
      authToken: AuthService.token.isNotEmpty ? AuthService.token : null,
    );

```

```

}

// =====
// 文件: services/consult_service.dart
// 原始行数: 242 | 保留行数: 120
// =====
import '../core/constants.dart';
import '../data/models/consult_session.dart';
import 'api_service.dart';
import 'auth_service.dart';

String _getCurrentDateInfo() {
  final now = DateTime.now();
  final weekday = ['一', '二', '三', '四', '五', '六', '日'][now.weekday - 1];
  return '${now.year}年${now.month}月${now.day}日(周$weekday)';
}

String _normalizeCategoryText(String text) {
  return text
    .trim()
    .replaceAll(RegExp(r'\s+'), '')
    .replaceAll(RegExp(r'^0-9A-Za-z\u4e00-\u9fff'), '');
}

String? _pickCategoryFromText(String? text, Iterable<String> categories) {
  if (text == null) return null;
  final normalized = _normalizeCategoryText(text);
  if (normalized.isEmpty) return null;
  for (final category in categories) {
    final normalizedCategory = _normalizeCategoryText(category);
    if (normalizedCategory.isEmpty) continue;
    if (normalized == normalizedCategory ||
        normalized.contains(normalizedCategory) ||
        normalizedCategory.contains(normalized)) {
      return category;
    }
  }
  return null;
}

MapEntry<String, double>? _findRecentCategoryAmount(
  Map<String, double> totals,
  String category,
) {
  final direct = totals[category];
  if (direct != null && direct > 0) {
    return MapEntry(category, direct);
  }
  final normalizedTarget = _normalizeCategoryText(category);

```

```

if (normalizedTarget.isEmpty) return null;

String? matchedKey;
var sum = 0.0;
for (final entry in totals.entries) {
    final normalizedKey = _normalizeCategoryText(entry.key);
    if (normalizedKey.isEmpty) continue;
    final matched =
        normalizedKey == normalizedTarget ||
        normalizedKey.contains(normalizedTarget) ||
        normalizedTarget.contains(normalizedKey);
    if (!matched) continue;
    matchedKey ??= entry.key;
    sum += entry.value;
}

if (sum <= 0) return null;
return MapEntry(matchedKey ?? category, sum);
}

Future<String?> classifyPurchaseIntentByAi(String userText) async {
    final text = userText.trim();
    if (text.isEmpty) return null;

    try {
        final response = await ApiService.chatCompletions(
            messages: <Map<String, String>>[
                <String, String>{
                    'role': 'user',
                    'content': '''用户说想买/想花: $text
请从以下分类中只返回一个分类名：餐饮、交通、购物、娱乐、居家、医疗、教育、其他。
只回复分类名，不要解释。'''
                },
            ],
            authToken: AuthService.token.isNotEmpty ? AuthService.token : null,
        );

        final cleaned = response.trim().replaceAll(RegExp(r'[■□"'"`~\s]+'), '');
        final picked = _pickCategoryFromText(
            cleaned,
            AppConstants.expenseCategories,
        );
        if (picked != null) return picked;

        return _pickCategoryFromText(response, AppConstants.expenseCategories);
    } catch (_) {
        return null;
    }
}

```

```

Stream<ConsultStreamChunk> consultStream({
  required List<Map<String, String>> conversationHistory,
  Map<String, double>? recentCategorySpending,
  bool enableAgentSteps = true,
  String? intentAnalysisTargetText,
  String? preclassifiedIntentCategory,
}) async* {
  final dateInfo = _getCurrentDateInfo();
  Map<String, double>? filteredSpending;
  const stepIntent = '分析购买意图';
  const stepRecent = '同类近30天支出';

  final userMessages = conversationHistory.where(
    (x) => (x['role'] ?? '') == 'user',
  );
  final firstUserContent = userMessages.isEmpty
    ? null
    : (userMessages.first['content'] ?? '');
  final analysisTarget = (intentAnalysisTargetText ?? firstUserContent ?? '')
    .trim();
  final needsAgentSteps = enableAgentSteps && analysisTarget.isNotEmpty;

  if (needsAgentSteps) {
    yield ConsultStreamChunk(
      agentSteps: <AgentStep>[const AgentStep(stepIntent)],
    );
  }

  // =====
  // 文件: services/auth_service.dart
  // 原始行数: 142 | 保留行数: 100
  // =====
  import 'dart:convert';
  import 'package:flutter/foundation.dart';
  import 'package:http/http.dart' as http;
  import 'package:shared_preferences/shared_preferences.dart';
  import 'api_service.dart';

  class AuthService {
    static const _kToken = 'auth_token';
    static const _kUserId = 'auth_user_id';
    static const _kPhone = 'auth_phone';
    static const _kLoggedIn = 'auth_logged_in';

    static String _token = '';
    static String _userId = '';
    static String _phone = '';
    static bool _loggedIn = false;

    static bool get isLoggedIn => _loggedIn;

```



```

static String get token => _token;
static String get userId => _userId;
static String get phone => _phone;

static VoidCallback? onAuthStateChanged;

static Future<void> init() async {
  final prefs = await SharedPreferences.getInstance();
  _token = (prefs.getString(_kToken) ?? '').trim();
  _userId = (prefs.getString(_kUserId) ?? '').trim();
  _phone = (prefs.getString(_kPhone) ?? '').trim();
  final persistedLoggedIn = prefs.getBool(_kLoggedIn) ?? false;
  _loggedIn = _token.isNotEmpty && (persistedLoggedIn || _userId.isNotEmpty);
  debugPrint('[Auth] init: loggedIn=$_loggedIn');
}

static Future<AuthResult> sendSmsCode(String phone) async {
  final baseUrl = ApiService.proxyUrl.trim();
  if (baseUrl.isEmpty) return AuthResult(success: false, error: '服务未配置');
  try {
    final resp = await http
      .post(
        Uri.parse('$baseUrl/auth/sms/send'),
        headers: ApiService.headers(),
        body: jsonEncode({'phone': phone}),
      )
      .timeout(const Duration(seconds: 15));
    final json = jsonDecode(resp.body);
    if (resp.statusCode == 200 && json['success'] == true) {
      return AuthResult(success: true);
    }
    return AuthResult(
      success: false,
      error: json['message'] ?? json['error'] ?? '发送失败',
    );
  } catch (e) {
    debugPrint('[Auth] sendSmsCode: $e');
    return AuthResult(success: false, error: '网络错误');
  }
}

static Future<AuthResult> loginWithSmsCode(String phone, String code) async {
  final baseUrl = ApiService.proxyUrl.trim();
  if (baseUrl.isEmpty) return AuthResult(success: false, error: '服务未配置');
  try {
    final headers = Map<String, String>.from(ApiService.headers());
    headers['X-Device-Id'] = ApiService.deviceId;
    final resp = await http
      .post(
        Uri.parse('$baseUrl/auth/sms/verify'),

```

```

        headers: headers,
        body: jsonEncode({'phone': phone, 'code': code}),
      )
      .timeout(const Duration(seconds: 15));
final json = jsonDecode(resp.body);
if (resp.statusCode == 200 && json['token'] != null) {
  _token = json['token'];
  _userId = json['userId'] ?? '';
  _phone = json['phone'] ?? phone;
  _loggedIn = true;
  final prefs = await SharedPreferences.getInstance();
  await prefs.setString(_kToken, _token);
  await prefs.setString(_kUserId, _userId);
  await prefs.setString(_kPhone, _phone);
  await prefs.setBool(_kLoggedIn, true);
  onAuthStateChanged?.call();
  return AuthResult(
    success: true,
    balance: (json['balance'] as num?)?.toInt(),
    initialGrantedThisTime: json['initialGrantedThisTime'] == true,
    initialGrantPoints: (json['initialGrantPoints'] as num?)?.toInt(),
  );
}
return AuthResult(
  success: false,
  error: json['message'] ?? json['error'] ?? '登录失败',
);
} catch (e) {
  debugPrint('[Auth] login: $e');
  return AuthResult(success: false, error: '网络错误');
}
}

// =====
// 文件: services/notification_service.dart
// 原始行数: 388 | 保留行数: 120
// =====
import 'dart:convert';

import 'package:flutter/foundation.dart';
import 'package:flutter_local_notifications/flutter_local_notifications.dart';
import 'package:shared_preferences/shared_preferences.dart';
import 'package:timezone/data/latest.dart' as tz_data;
import 'package:timezone/timezone.dart' as tz;

String _normalizeReminderTime(String value) {
  final parts = value.split(':');
  final hour = parts.isNotEmpty ? int.tryParse(parts[0]) ?? 13 : 13;
  final minute = parts.length > 1 ? int.tryParse(parts[1]) ?? 0 : 0;
  final safeHour = hour.clamp(0, 23);

```

```

    final safeMinute = minute.clamp(0, 59);
    return '${safeHour.toString().padLeft(2, '0')}:${safeMinute.toString().padLeft(2, '0')}';
}

class ReminderEntry {
  const ReminderEntry({required this.time, required this.enabled});

  final String time;
  final bool enabled;

  ReminderEntry copyWith({String? time, bool? enabled}) {
    return ReminderEntry(
      time: time ?? this.time,
      enabled: enabled ?? this.enabled,
    );
  }

  Map<String, dynamic> toJson() {
    return <String, dynamic>{
      'time': _normalizeReminderTime(time),
      'enabled': enabled,
    };
  }

  factory ReminderEntry.fromJson(Map<String, dynamic> json) {
    final rawEnabled = json['enabled'];
    final parsedEnabled = rawEnabled is bool
      ? rawEnabled
      : rawEnabled?.toString().toLowerCase() == 'true';
    return ReminderEntry(
      time: _normalizeReminderTime(json['time']?.toString() ?? '13:00'),
      enabled: parsedEnabled,
    );
  }
}

class NotificationService {
  NotificationService._();

  static const _kEntries = 'reminder_entries';
  static const _kLegacyEnabled = 'reminder_enabled';
  static const _kLegacyTimes = 'reminder_times';

  static const _channelId = 'airmoney_reminder_v3';
  static const _channelName = '记账提醒';
  static const _notificationIcon = 'ic_stat_airmoney';
  static const _notificationIdBase = 1000;
  static const _maxReminderCount = 12;

  static final FlutterLocalNotificationsPlugin _plugin =

```

```

FlutterLocalNotificationsPlugin();

static bool _initialized = false;
static Future<void>? _initializing;

static List<ReminderEntry> _defaultEntries() {
  return const <ReminderEntry>[
    ReminderEntry(time: '13:00', enabled: false),
    ReminderEntry(time: '20:00', enabled: false),
  ];
}

static List<ReminderEntry> _sanitizeEntries(List<ReminderEntry> entries) {
  return entries
    .take(24)
    .map(
      (entry) => ReminderEntry(
        time: _normalizeReminderTime(entry.time),
        enabled: entry.enabled,
      ),
    )
    .toList();
}

static Future<List<ReminderEntry>> _readEntries(
  SharedPreferences prefs,
) async {
  final raw = prefs.getString(_kEntries);
  if (raw != null && raw.isNotEmpty) {
    try {
      final decoded = jsonDecode(raw);
      if (decoded is List) {
        final parsed = <ReminderEntry>[];
        for (final item in decoded) {
          if (item is Map<String, dynamic>) {
            parsed.add(ReminderEntry.fromJson(item));
          } else if (item is Map) {
            parsed.add(
              ReminderEntry.fromJson(item.cast<String, dynamic>()),
            );
          }
        }
        final sanitized = _sanitizeEntries(parsed);
        return sanitized;
      }
    } catch (_) {
      // Ignore malformed payload and fallback.
    }
  }
}

```

```

final legacyCsv = prefs.getString(_kLegacyTimes);
final legacyTimes = legacyCsv == null || legacyCsv.trim().isEmpty
? <String>[]
: legacyCsv
    .split(',')
    .map((value) => value.trim())
    .where((value) => value.isNotEmpty)

// =====
// 文件: services/consult_session_storage.dart
// 原始行数: 155 | 保留行数: 100
// =====
import 'dart:convert';
import 'package:shared_preferences/shared_preferences.dart';
import '../data/models/consult_session.dart';

const _kSessions = 'consult_sessions';
const _kCurrentSessionId = 'consult_current_session_id';
const _kMaxSessions = 50;

class ConsultSessionStorage {
  static List<ConsultSession> _sessions = [];
  static String? _currentSessionId;

  static List<ConsultSession> get sessions => List.unmodifiable(_sessions);

  static String? get currentSessionId => _currentSessionId;

  static Future<void> load() async {
    final prefs = await SharedPreferences.getInstance();
    final listStr = prefs.getString(_kSessions);
    if (listStr != null && listStr.isNotEmpty) {
      try {
        final list = jsonDecode(listStr) as List<dynamic>;
        _sessions = list
          ?.map((e) => ConsultSession.fromJson(Map<String, dynamic>.from(e as Map)))
          .toList() ??
          [];
        _sessions.sort((a, b) => b.updatedAt.compareTo(a.updatedAt));
      } catch (_) {
        _sessions = [];
      }
    } else {
      _sessions = [];
    }
    _currentSessionId = prefs.getString(_kCurrentSessionId);
  }

  static Future<void> save() async {
    final prefs = await SharedPreferences.getInstance();

```

```

    final list = _sessions.map((s) => s.toJson()).toList();
    await prefs.setString(_kSessions, jsonEncode(list));
    final cid = _currentSessionId;
    if (cid != null) {
        await prefs.setString(_kCurrentSessionId, cid);
    } else {
        await prefs.remove(_kCurrentSessionId);
    }
}

static Future<void> setCurrentSession(String? id) async {
    _currentSessionId = id;
    await save();
}

static ConsultSession? getCurrentSession() {
    if (_currentSessionId == null) return null;
    try {
        return _sessions.firstWhere((s) => s.id == _currentSessionId);
    } catch (_) {
        return null;
    }
}

/// 获取或创建当前会话，进入页面时调用（需先 await load()）
static Future<ConsultSession> ensureCurrentSession() async {
    final cur = getCurrentSession();
    if (cur != null) return cur;
    return createNewSession();
}

/// 新建会话
static Future<ConsultSession> createNewSession() async {
    final newOne = ConsultSession(
        id: DateTime.now().millisecondsSinceEpoch.toString(),
        title: '新对话',
        messages: [],
        updatedAt: DateTime.now(),
    );
    _sessions.insert(0, newOne);
    if (_sessions.length > _kMaxSessions) {
        _sessions = _sessions.take(_kMaxSessions).toList();
    }
    _currentSessionId = newOne.id;
    await save();
    return newOne;
}

/// 切换会话
static Future<ConsultSession?> switchToSession(String id) async {

```

```

    try {
      final s = _sessions.firstWhere((s) => s.id == id);
      _currentSessionId = id;
      await save();
      return s;
    } catch (_) {
      return null;
    }
  }

  /// 追加消息并保存
  static Future<void> appendMessage(String sessionId, String role, String content, {String? reasonin

// =====
// 文件: presentation/widgets/reminder_settings_sheet.dart
// 原始行数: 295 | 保留行数: 120
// =====
import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';

import '../core/theme/app_colors.dart';
import '../services/notification_service.dart';

/// 通知提醒设置 BottomSheet
class ReminderSettingsSheet extends StatefulWidget {
  const ReminderSettingsSheet({super.key});

  static Future<bool> show(BuildContext context) async {
    final result = await showModalBottomSheet<bool>({
      context: context,
      isScrollControlled: true,
      shape: const RoundedRectangleBorder(
        borderRadius: BorderRadius.vertical(top: Radius.circular(16)),
      ),
      builder: (_) => const ReminderSettingsSheet(),
    });
    return result ?? false;
  }

  @override
  State<ReminderSettingsSheet> createState() => _ReminderSettingsSheetState();
}

class _ReminderSettingsSheetState extends State<ReminderSettingsSheet> {
  List<ReminderEntry> _entries = const <ReminderEntry>[];
  bool _loading = true;
  bool _saving = false;

  bool get _hasEnabledEntry => _entries.any((entry) => entry.enabled);

```

```

@Override
void initState() {
  super.initState();
  _load();
}

Future<void> _load() async {
  final entries = await NotificationService.getReminderEntries();
  if (!mounted) return;
  setState(() {
    _entries = List<ReminderEntry>.from(entries);
    _loading = false;
  });
}

Future<void> _save() async {
  if (_saving) return;
  setState(() => _saving = true);

  try {
    if (_hasEnabledEntry) {
      final granted = await NotificationService.requestPermissionIfNeeded();
      if (!granted) {
        throw Exception('未授予通知权限，请先在系统设置中允许通知');
      }
    }

    await NotificationService.setReminderEntries(_entries);

    if (!mounted) return;
    Navigator.of(context).pop(true);
  } catch (e) {
    if (!mounted) return;
    final msg = e.toString().replaceAll('Exception: ', '');
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('保存失败: $msg')),
    );
  } finally {
    if (mounted) {
      setState(() => _saving = false);
    }
  }
}

String _suggestNextTime() {
  if (_entries.isEmpty) return '13:00';
  final last = _entries.last.time.split(':');
  final hour = last.isNotEmpty ? int.tryParse(last[0]) ?? 13 : 13;
  final minute = last.length > 1 ? int.tryParse(last[1]) ?? 0 : 0;
  final nextHour = (hour + 1) % 24;

```



```

    return '${nextHour.toString().padLeft(2, '0')}:${minute.toString().padLeft(2, '0')}';
  }

  void _addEntry() {
    if (_entries.length >= 12) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('最多添加 12 个提醒时间')),
      );
      return;
    }
    setState(() {
      _entries = List<ReminderEntry>.from(_entries)
        ..add(ReminderEntry(time: _suggestNextTime(), enabled: false));
    });
  }

  void _removeEntry(int index) {
    setState(() {
      final next = List<ReminderEntry>.from(_entries);
      next.removeAt(index);
      _entries = next;
    });
  }

  void _toggleEntry(int index, bool enabled) {
    setState(() {
      final next = List<ReminderEntry>.from(_entries);
      next[index] = next[index].copyWith(enabled: enabled);
      _entries = next;
    });
  }

  Future<void> _pickTime(int index) async {
    final parts = _entries[index].time.split(':');
    final hour = int.tryParse(parts[0]) ?? 13;
    final minute = int.tryParse(parts.length > 1 ? parts[1] : '0') ?? 0;

    // =====
    // 文件: presentation/widgets/wallet_sheet.dart
    // 原始行数: 275 | 保留行数: 120
    // =====
    import 'package:flutter/material.dart';
    import 'package:intl/intl.dart';
    import '../core/theme/app_colors.dart';
    import '../services/api_service.dart';
    import '../services/auth_service.dart';
    import '../pages/auth/login_page.dart';

    class WalletSheet extends StatelessWidget {

```

```

final int balance;
final VoidCallback onBalanceChanged;

const WalletSheet({
  super.key,
  required this.balance,
  required this.onBalanceChanged,
});

static Future<void> show(BuildContext context, int balance, VoidCallback onBalanceChanged) {
  return showModalBottomSheet(
    context: context,
    shape: const RoundedRectangleBorder(
      borderRadius: BorderRadius.vertical(top: Radius.circular(24)),
    ),
    builder: (_) => WalletSheet(balance: balance, onBalanceChanged: onBalanceChanged),
  );
}

@override
Widget build(BuildContext context) {
  return _WalletSheetContent(
    initialBalance: balance,
    onBalanceChanged: onBalanceChanged,
  );
}

class _WalletSheetContent extends StatefulWidget {
  final int initialBalance;
  final VoidCallback onBalanceChanged;

  const _WalletSheetContent({
    required this.initialBalance,
    required this.onBalanceChanged,
  });

  @override
  State<_WalletSheetContent> createState() => _WalletSheetContentState();
}

class _WalletSheetContentState extends State<_WalletSheetContent> {
  int _currentBalance = 0;
  bool _checkedInToday = false;
  bool _checkinBusy = false;
  int _checkinPoints = 5000;
  int _initialGrantPoints = 500000;

  @override
  void initState() {

```

```

    super.initState();
    _currentBalance = widget.initialBalance;
    _loadCheckinStatus();
    _loadConfig();
  }

  Future<void> _loadConfig() async {
    try {
      final resp = await ApiService.getConfig();
      if (mounted) {
        setState(() {
          _checkinPoints = (resp['checkin_points'] as num?)?.toInt() ?? 5000;
          _initialGrantPoints = (resp['initial_grant_points'] as num?)?.toInt() ?? 500000;
        });
      }
    } catch (_) {}
  }

  Future<void> _loadCheckinStatus() async {
    try {
      final status = await ApiService.getCheckinStatus(
        authToken: AuthService.token.isNotEmpty ? AuthService.token : null,
      );
      if (mounted) setState(() => _checkedInToday = status);
    } catch (_) {}
  }

  Future<void> _refreshBalance() async {
    try {
      final b = await ApiService.getPointsBalance(
        authToken: AuthService.token.isNotEmpty ? AuthService.token : null,
      );
      if (mounted) setState(() => _currentBalance = b);
    } catch (_) {}
  }

  Future<void> _doCheckin() async {
    if (_checkedInToday || _checkinBusy) return;
    setState(() => _checkinBusy = true);
    try {
      final result = await ApiService.checkin(
        authToken: AuthService.token.isNotEmpty ? AuthService.token : null,
      );
      if (mounted) {
        setState(() {
          _checkedInToday = true;
          if (result.balance != null) {
            _currentBalance = result.balance!;
          } else {
            _currentBalance += result.points;
          }
        });
      }
    }
  }

```

```
    }  
  });  
  widget.onBalanceChanged();  
  if (result.points > 0) {  
    ScaffoldMessenger.of(context).showSnackBar(  
      SnackBar(  
        content: Text('签到成功! +${NumberFormat('#,###').format(result.points)} 积分'),  
        behavior: SnackBarBehavior.floating,  
      ),  
    );  
  }  
}
```